Stacks & Queues Lab

Data Structures & Algorithms

Lab 03 — Implementing Stacks & Queues from Scratch

Duration: 2 hours (4 challenges × 30 minutes each)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (2 min)
2. **Your Implementation Time** (20 min)
3. **Pseudocode Review** (3 min)
4. **Solution Walkthrough** (5 min)
5. **Code Explanation** (5 min)

Total per challenge: ~35 minutes

Today's Topics

Stack Using Arrays

- Fixed-size stack with `push`, `pop`, `peek`
- Overflow and underflow handling

Stack Using Linked List

- Dynamic stack with no size limit
- Proper memory management with `malloc` / `free`

Queue Using Arrays (Circular Queue)

- Solving the "wasted space" problem with modular arithmetic
- Enqueue, dequeue, and wrap-around logic

Queue Using Linked List

- Dynamic queue with front and rear pointers
- Edge cases with single-element queues

Challenge 1

Stack Using Arrays

Challenge 1: Problem Statement

Task: Implement a **Stack** data structure using a fixed-size array. Your stack should support all fundamental operations and handle error conditions gracefully.

Requirements:

- Define a `Stack` struct with an array of size `MAX = 5`, and a `top` variable
- Implement `push(stack, value)` — add element to the top
- Implement `pop(stack)` — remove and return the top element
- Implement `peek(stack)` — view the top element without removing
- Implement `isEmpty(stack)` and `isFull(stack)`
- Handle **overflow** (push on full stack) and **underflow** (pop on empty stack)

Time: 20 minutes

Challenge 1: Pseudocode

```
ALGORITHM ArrayStack

  1. Define Stack structure:
     - array of MAX integers
     - top variable initialized to -1

  2. push(stack, value):
     - If top == MAX - 1: print "Overflow!", return
     - Increment top
     - Set arr[top] = value

  3. pop(stack):
     - If top == -1: print "Underflow!", return -1
     - Save value = arr[top]
     - Decrement top
     - Return value

  4. peek(stack):
     - If top == -1: print "Empty!", return -1
     - Return arr[top]

  5. isEmpty(stack): return top == -1
  6. isFull(stack): return top == MAX - 1
END ALGORITHM
```

Challenge 1: Solution Code

```c
#include <stdio.h>

#define MAX 5

struct Stack {
    int arr[MAX];
    int top;
};

// Initialize stack
void initStack(struct Stack* s) {
    s->top = -1;
}

// Check if stack is empty
int isEmpty(struct Stack* s) {
    return s->top == -1;
}

// Check if stack is full
int isFull(struct Stack* s) {
    return s->top == MAX - 1;
}
```

Challenge 1: Solution Code (continued)

```c
// Push element onto stack
void push(struct Stack* s, int value) {
    if (isFull(s)) {
        printf("Stack Overflow! Cannot push %d\n", value);
        return;
    }
    s->top++;
    s->arr[s->top] = value;
    printf("Pushed %d (top = %d)\n", value, s->top);
}

// Pop element from stack
int pop(struct Stack* s) {
    if (isEmpty(s)) {
        printf("Stack Underflow! Cannot pop\n");
        return -1;
    }
    int value = s->arr[s->top];
    s->top--;
    printf("Popped %d (top = %d)\n", value, s->top);
    return value;
}

// Peek at top element
int peek(struct Stack* s) {
    if (isEmpty(s)) {
        printf("Stack is empty!\n");
        return -1;
    }
    return s->arr[s->top];
}
```

Challenge 1: Solution Code (Test Cases)

```c
int main() {
    struct Stack s;
    initStack(&s);

    printf("=== Array Stack Demo (MAX = %d) ===\n\n", MAX);

    // Push elements
    push(&s, 10);
    push(&s, 20);
    push(&s, 30);
    push(&s, 40);
    push(&s, 50);
    push(&s, 60);   // Overflow!

    printf("\nPeek: %d\n\n", peek(&s));

    // Pop all elements
    while (!isEmpty(&s)) {
        pop(&s);
    }

    pop(&s);  // Underflow!

    return 0;
}
```

Challenge 1: Code Explanation

Part 1: The `top` Variable

```
struct Stack {
    int arr[MAX];
    int top;          // Index of topmost element
};
```

Why `top = -1` for empty?

- `top` always points to the current topmost element
- When the stack is empty, there is no topmost element
- Using `-1` as a sentinel value makes the logic clean:
  - Push: `top++` then write → first push makes `top = 0`
  - Pop: read then `top--` → last pop makes `top = -1`

```
Empty:  top = -1     One item:  top = 0     Two items:  top = 1
[ ][ ][ ][ ][ ]      [10][ ][ ][ ][ ]       [10][20][ ][ ][ ]
                        ^                              ^
                      top                            top
```

Challenge 1: Code Explanation

Part 2: Overflow and Underflow

Overflow happens when you try to push onto a full stack:

```
Stack is full:  top = MAX - 1 = 4
[10][20][30][40][50]     push(60) → ERROR!
                  ^
                 top
```

Underflow happens when you try to pop from an empty stack:

```
Stack is empty:  top = -1
[ ][ ][ ][ ][ ]          pop() → ERROR!
```

Why check before every operation?

Without checks, pushing beyond `MAX` writes to memory *outside* the array — this is a **buffer overflow**, one of the most dangerous bugs in C programming (and a common source of security vulnerabilities!).

Challenge 1: Code Explanation

Part 3: Why Pass a Pointer to Stack?

```
void push(struct Stack* s, int value) {
    s->top++;                // Modifies the original struct
    s->arr[s->top] = value; // Modifies the original array
}
```

Why `struct Stack*` and not `struct Stack`?

- In C, function arguments are **passed by value** (copied)
- If we pass `struct Stack s`, the function gets a copy
- Changes to `top` and `arr` would only affect the copy
- Passing a pointer lets us modify the **original** struct

The `->` operator:

- `s->top` is shorthand for `(*s).top`
- Dereference the pointer, then access the field

Challenge 1: Code Explanation

Part 4: Complexity Analysis

Operation	Time Complexity	Why?
`push()`	O(1)	Increment index, write value
`pop()`	O(1)	Read value, decrement index
`peek()`	O(1)	Read value at index
`isEmpty()`	O(1)	Compare `top` with -1
`isFull()`	O(1)	Compare `top` with MAX-1

Every operation is O(1) — constant time regardless of stack size.

Space Complexity: O(MAX) — we always allocate `MAX` slots, even if the stack is almost empty. This is the trade-off of using a fixed-size array.

Challenge 2

Stack Using Linked List

Challenge 2: Problem Statement

Task: Implement a **Stack** using a singly linked list. The top of the stack should be the head of the linked list, so push and pop both operate at the head.

Requirements:

- Define a `Node` struct with `data` and `next`
- Implement `push(top, value)` — create a new node, insert at head
- Implement `pop(top)` — remove and return head node's data, free the memory
- Implement `peek(top)` — view head node's data without removing
- Implement `isEmpty(top)`
- No fixed size limit — stack grows dynamically
- **Free memory** when nodes are popped (no memory leaks!)

Time: 20 minutes

Challenge 2: Pseudocode

```
ALGORITHM LinkedListStack

  1. Define Node structure:
     - int data
     - pointer to next Node

  2. push(top, value):
     - Create new node with malloc
     - Set newNode->data = value
     - Set newNode->next = current top
     - Update top = newNode

  3. pop(top):
     - If top == NULL: print "Underflow!", return -1
     - Save value = top->data
     - Save temp = top
     - Move top = top->next
     - Free temp
     - Return value

  4. peek(top):
     - If top == NULL: print "Empty!", return -1
     - Return top->data

  5. isEmpty(top): return top == NULL
END ALGORITHM
```

Challenge 2: Solution Code

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Push element onto stack (insert at head)
void push(struct Node** top, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;
    newNode->next = *top;
    *top = newNode;
    printf("Pushed %d\n", value);
}
```

Challenge 2: Solution Code (continued)

```c
// Pop element from stack (remove from head)
int pop(struct Node** top) {
    if (*top == NULL) {
        printf("Stack Underflow!\n");
        return -1;
    }
    struct Node* temp = *top;
    int value = temp->data;
    *top = (*top)->next;
    free(temp);
    printf("Popped %d\n", value);
    return value;
}

// Peek at top element
int peek(struct Node* top) {
    if (top == NULL) {
        printf("Stack is empty!\n");
        return -1;
    }
    return top->data;
}

// Check if stack is empty
int isEmpty(struct Node* top) {
    return top == NULL;
}
```

Challenge 2: Solution Code (continued)

```c
// Display the stack contents
void displayStack(struct Node* top) {
    printf("Stack (top → bottom): ");
    struct Node* current = top;
    while (current != NULL) {
        printf("[%d]", current->data);
        if (current->next != NULL) printf(" → ");
        current = current->next;
    }
    if (top == NULL) printf("[EMPTY]");
    printf("\n");
}

// Free all remaining nodes
void freeStack(struct Node** top) {
    while (*top != NULL) {
        struct Node* temp = *top;
        *top = (*top)->next;
        free(temp);
    }
    printf("Stack memory freed.\n");
}
```

Challenge 2: Solution Code (Test Cases)

```c
int main() {
    struct Node* top = NULL;  // Empty stack

    printf("=== Linked List Stack Demo ===\n\n");

    push(&top, 10);
    push(&top, 20);
    push(&top, 30);
    push(&top, 40);
    displayStack(top);

    printf("\nPeek: %d\n\n", peek(top));

    pop(&top);
    pop(&top);
    displayStack(top);

    push(&top, 50);
    push(&top, 60);
    displayStack(top);

    // Clean up all remaining memory
    freeStack(&top);
    displayStack(top);

    return 0;
}
```

Challenge 2: Code Explanation

Part 1: Why `struct Node**` (Double Pointer)?

```
void push(struct Node** top, int value) {
    // ...
    *top = newNode;  // Changes what 'top' points to in the caller
}
```

Why not just `struct Node*`?

```
Single pointer (WRONG):                Double pointer (CORRECT):
  main's top → [old head]                main's top → [old head]
  push gets a COPY of top                           ↑
  Modifying the copy doesn't             push gets pointer TO main's top
  change main's top!                     *top = newNode changes main's top!
```

Rule of thumb in C: If a function needs to change a pointer in the caller, pass a **pointer to that pointer**.

Challenge 2: Code Explanation

Part 2: Push Operation — Step by Step

```
Before: top → [30|*] → [20|*] → [10|NULL]

push(top, 40):
  Step 1: malloc new node          newNode → [40| ? ]
  Step 2: newNode->data = 40       newNode → [40| ? ]
  Step 3: newNode->next = *top     newNode → [40| *] → [30|*] → [20|*] → [10|NULL]
  Step 4: *top = newNode

After:  top → [40|*] → [30|*] → [20|*] → [10|NULL]
```

Key insight: Push is **insert-at-head** on a linked list. We already practiced this in the linked list classes!

Challenge 2: Code Explanation

Part 3: Pop Operation — The Memory Danger

```
int pop(struct Node** top) {
    struct Node* temp = *top;      // 1. Save pointer to current top
    int value = temp->data;        // 2. Save the data
    *top = (*top)->next;           // 3. Move top one step forward
    free(temp);                    // 4. Free the old top node
    return value;                  // 5. Return the saved data
}
```

Why save `temp` before freeing?

❌ Without temp:

```
int value = (*top)->data;
free(*top);             // Node is freed!
*top = (*top)->next;    // DANGER: accessing freed memory! (Undefined behaviour)
```

✅ **With temp:** We save the pointer, update `top`, then safely free.

Challenge 2: Code Explanation

Part 4: Array Stack vs. Linked List Stack

Feature	Array Stack	Linked List Stack
Size	Fixed (`MAX`)	Dynamic
Overflow	When `top == MAX-1`	Only if `malloc` fails
Memory per element	Just the data	Data + pointer (8 extra bytes)
Cache performance	Better (contiguous memory)	Worse (scattered nodes)
Code complexity	Simpler	Slightly more (pointers)
Memory cleanup	Nothing needed	Must free every node

Both have O(1) for all operations. The choice depends on whether you prioritize simplicity (array) or flexibility (linked list).

Challenge 3

Queue Using Arrays (Circular Queue)

Challenge 3: Problem Statement

Task: Implement a **Circular Queue** using an array. A circular queue wraps around when the rear reaches the end, reusing space freed by dequeue operations.

Requirements:

- Define a `CircularQueue` struct with an array of size `MAX = 5`, and `front`, `rear`, and `count` variables
- Implement `enqueue(queue, value)` — add element at the rear
- Implement `dequeue(queue)` — remove and return element from the front
- Implement `peek(queue)` — view front element without removing
- Implement `isEmpty(queue)` and `isFull(queue)`
- Use **modular arithmetic** (`% MAX`) for wrap-around
- Display the queue state after each operation

Time: 20 minutes

Challenge 3: Pseudocode

```
ALGORITHM CircularQueue

  1. Define Queue structure:
     - array of MAX integers
     - front = 0, rear = -1, count = 0

  2. enqueue(queue, value):
     - If count == MAX: print "Overflow!", return
     - rear = (rear + 1) % MAX
     - arr[rear] = value
     - Increment count

  3. dequeue(queue):
     - If count == 0: print "Underflow!", return -1
     - Save value = arr[front]
     - front = (front + 1) % MAX
     - Decrement count
     - Return value

  4. peek(queue):
     - If count == 0: print "Empty!", return -1
     - Return arr[front]

  5. isEmpty(queue): return count == 0
  6. isFull(queue): return count == MAX
END ALGORITHM
```

Challenge 3: Solution Code

```c
#include <stdio.h>

#define MAX 5

struct CircularQueue {
    int arr[MAX];
    int front;
    int rear;
    int count;
};

// Initialize queue
void initQueue(struct CircularQueue* q) {
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}

int isEmpty(struct CircularQueue* q) {
    return q->count == 0;
}

int isFull(struct CircularQueue* q) {
    return q->count == MAX;
}
```

Challenge 3: Solution Code (continued)

```c
// Enqueue element at rear
void enqueue(struct CircularQueue* q, int value) {
    if (isFull(q)) {
        printf("Queue Overflow! Cannot enqueue %d\n", value);
        return;
    }
    q->rear = (q->rear + 1) % MAX;
    q->arr[q->rear] = value;
    q->count++;
    printf("Enqueued %d at index %d\n", value, q->rear);
}

// Dequeue element from front
int dequeue(struct CircularQueue* q) {
    if (isEmpty(q)) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int value = q->arr[q->front];
    printf("Dequeued %d from index %d\n", value, q->front);
    q->front = (q->front + 1) % MAX;
    q->count--;
    return value;
}

// Peek at front element
int peek(struct CircularQueue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q->arr[q->front];
}
```

Challenge 3: Solution Code (Display)

```c
// Display queue contents (front to rear)
void displayQueue(struct CircularQueue* q) {
    if (isEmpty(q)) {
        printf("Queue: [EMPTY]\n");
        return;
    }

    printf("Queue (front → rear): ");
    int idx = q->front;
    for (int i = 0; i < q->count; i++) {
        printf("[%d]", q->arr[idx]);
        if (i < q->count - 1) printf(" → ");
        idx = (idx + 1) % MAX;
    }
    printf("  (count=%d, front=%d, rear=%d)\n",
           q->count, q->front, q->rear);
}
```

Challenge 3: Solution Code (Test Cases)

```c
int main() {
    struct CircularQueue q;
    initQueue(&q);

    printf("=== Circular Queue Demo (MAX = %d) ===\n\n", MAX);

    // Fill the queue
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    enqueue(&q, 40);
    enqueue(&q, 50);
    displayQueue(&q);

    enqueue(&q, 60);  // Overflow!

    printf("\n--- Dequeue two elements ---\n");
    dequeue(&q);
    dequeue(&q);
    displayQueue(&q);

    printf("\n--- Enqueue two more (wrap-around!) ---\n");
    enqueue(&q, 60);
    enqueue(&q, 70);
    displayQueue(&q);

    printf("\nPeek: %d\n", peek(&q));

    return 0;
}
```

Challenge 3: Code Explanation

Part 1: The Problem with Linear Queues

In a simple linear queue, after several enqueue/dequeue operations, `front` moves forward and leaves **wasted space** at the beginning:

```
After 3 enqueues + 2 dequeues:

[ _ ][ _ ][30][ _ ][ _ ]    front=2, rear=2
  ^    ^
  wasted space — can't use!

After 3 more enqueues:
[ _ ][ _ ][30][40][50]       rear=4 → OVERFLOW!
  ^    ^
  Two slots unused but we can't reach them!
```

This is called "false overflow" — the array has space but the queue thinks it's full.

Challenge 3: Code Explanation

Part 2: Modular Arithmetic to the Rescue

The key trick: wrap the index around using `% MAX`

```
q->rear = (q->rear + 1) % MAX;
q->front = (q->front + 1) % MAX;
```

How it works for MAX = 5:

Current Index	`(index + 1) % 5`	Effect
0	1	Normal increment
1	2	Normal increment
2	3	Normal increment
3	4	Normal increment
4	**0**	**Wraps around!**

Challenge 3: Code Explanation

Part 3: Wrap-Around Trace

```
Step 1: Enqueue 10, 20, 30, 40, 50 → Queue is FULL
[10][20][30][40][50]     front=0, rear=4, count=5

Step 2: Dequeue twice → removes 10 and 20
[ _ ][ _ ][30][40][50]   front=2, rear=4, count=3

Step 3: Enqueue 60 → rear = (4+1) % 5 = 0 → WRAPS AROUND!
[60][ _ ][30][40][50]    front=2, rear=0, count=4

Step 4: Enqueue 70 → rear = (0+1) % 5 = 1
[60][70][30][40][50]     front=2, rear=1, count=5
```

The queue now spans: index 2 → 3 → 4 → 0 → 1

Reading order (front to rear): 30, 40, 50, 60, 70 ✓

Challenge 3: Code Explanation

Part 4: Why Use `count` Instead of Comparing `front` and `rear`?

The classic problem: In some implementations, an empty queue and a full queue *both* have the same condition: `front == rear`!

```
Empty:  front = rear = 0      Full:  front = rear = 0
[  ][  ][  ][  ][  ]          [50][10][20][30][40]
 ^                             ^
 front & rear                  front & rear — ambiguous!
```

Using `count` eliminates this ambiguity:

- Empty: `count == 0`
- Full: `count == MAX`
- No confusion, ever!

Alternative: Some implementations sacrifice one slot (use `MAX - 1` capacity). Using `count` gives us full capacity.

Challenge 4

Queue Using Linked List

Challenge 4: Problem Statement

Task: Implement a **Queue** using a singly linked list. Maintain both a `front` and `rear` pointer so that enqueue (at rear) and dequeue (from front) are both O(1).

Requirements:

- Define a `Node` struct with `data` and `next`
- Define a `Queue` struct with `front` and `rear` pointers and a `count`
- Implement `enqueue(queue, value)` — create node, add at rear
- Implement `dequeue(queue)` — remove and return front node's data, free the memory
- Implement `peek(queue)` — view front element without removing
- Implement `isEmpty(queue)` and `displayQueue(queue)`
- Handle the edge case when the queue goes from one element to empty

Time: 20 minutes

Challenge 4: Pseudocode

```
ALGORITHM LinkedListQueue

  1. Define Node structure:
     - int data, pointer to next Node

  2. Define Queue structure:
     - pointer front (head), pointer rear (tail), int count

  3. enqueue(queue, value):
     - Create new node with malloc
     - newNode->next = NULL
     - If rear is NULL (empty queue):
       * front = rear = newNode
     - Else:
       * rear->next = newNode
       * rear = newNode
     - Increment count

  4. dequeue(queue):
     - If front == NULL: print "Underflow!", return -1
     - Save value = front->data
     - Save temp = front
     - front = front->next
     - If front == NULL: rear = NULL (queue is now empty)
     - Free temp, decrement count
     - Return value

  5. isEmpty(queue): return front == NULL
END ALGORITHM
```

Challenge 4: Solution Code

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Queue {
    struct Node* front;
    struct Node* rear;
    int count;
};

// Initialize queue
void initQueue(struct Queue* q) {
    q->front = NULL;
    q->rear = NULL;
    q->count = 0;
}
```

Challenge 4: Solution Code (continued)

```c
// Enqueue element at rear
void enqueue(struct Queue* q, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;

    if (q->rear == NULL) {
        // Queue was empty — new node is both front and rear
        q->front = newNode;
        q->rear = newNode;
    } else {
        // Add after current rear, then update rear
        q->rear->next = newNode;
        q->rear = newNode;
    }

    q->count++;
    printf("Enqueued %d (count = %d)\n", value, q->count);
}
```

Challenge 4: Solution Code (continued)

```c
// Dequeue element from front
int dequeue(struct Queue* q) {
    if (q->front == NULL) {
        printf("Queue Underflow!\n");
        return -1;
    }
    struct Node* temp = q->front;
    int value = temp->data;
    q->front = q->front->next;

    // If queue is now empty, update rear too
    if (q->front == NULL) {
        q->rear = NULL;
    }

    free(temp);
    q->count--;
    printf("Dequeued %d (count = %d)\n", value, q->count);
    return value;
}

// Peek at front element
int peek(struct Queue* q) {
    if (q->front == NULL) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q->front->data;
}

int isEmpty(struct Queue* q) {
    return q->front == NULL;
}
```

Challenge 4: Solution Code (Display & Cleanup)

```c
// Display queue contents
void displayQueue(struct Queue* q) {
    if (isEmpty(q)) {
        printf("Queue: [EMPTY]\n");
        return;
    }
    printf("Queue (front → rear): ");
    struct Node* current = q->front;
    while (current != NULL) {
        printf("[%d]", current->data);
        if (current->next != NULL) printf(" → ");
        current = current->next;
    }
    printf("  (count=%d)\n", q->count);
}

// Free all remaining nodes
void freeQueue(struct Queue* q) {
    while (q->front != NULL) {
        struct Node* temp = q->front;
        q->front = q->front->next;
        free(temp);
    }
    q->rear = NULL;
    q->count = 0;
    printf("Queue memory freed.\n");
}
```

Challenge 4: Solution Code (Test Cases)

```c
int main() {
    struct Queue q;
    initQueue(&q);

    printf("=== Linked List Queue Demo ===\n\n");

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    enqueue(&q, 40);
    displayQueue(&q);

    printf("\nPeek: %d\n\n", peek(&q));

    dequeue(&q);
    dequeue(&q);
    displayQueue(&q);

    printf("\n--- Enqueue more elements ---\n");
    enqueue(&q, 50);
    enqueue(&q, 60);
    displayQueue(&q);

    printf("\n--- Drain the queue ---\n");
    while (!isEmpty(&q)) {
        dequeue(&q);
    }
    dequeue(&q);  // Underflow!

    freeQueue(&q);
    return 0;
}
```

Challenge 4: Code Explanation

Part 1: Why Two Pointers (front and rear)?

```
With only front pointer:
  front → [10|*] → [20|*] → [30|NULL]

  Enqueue = insert at tail → must TRAVERSE entire list → O(n)!

With both front and rear:
  front → [10|*] → [20|*] → [30|NULL] ← rear

  Enqueue: rear->next = newNode; rear = newNode  → O(1)!
  Dequeue: temp = front; front = front->next     → O(1)!
```

Maintaining two pointers is what makes linked list queue efficient. Without `rear`, enqueue degrades to O(n).

Challenge 4: Code Explanation

Part 2: The Critical Edge Case — Single Element Queue

```
if (q->front == NULL) {
    q->rear = NULL;  // Queue is now empty!
}
```

Why is this necessary?

```
Before dequeue (1 element):
  front → [10|NULL] ← rear

After dequeue:
  front = NULL        ← correct
  rear still → [freed memory] ← DANGLING POINTER!
```

If we forget to set `rear = NULL`:

- `rear` points to freed memory (dangling pointer)
- Next `enqueue` would do `rear->next = newNode` → **Undefined behavior!**

Challenge 4: Code Explanation

Part 3: Enqueue Visualization

```
Start:  front = NULL, rear = NULL (empty)

enqueue(10):
  front → [10|NULL] ← rear        (first node is both front and rear)

enqueue(20):
  front → [10|*] → [20|NULL] ← rear

enqueue(30):
  front → [10|*] → [20|*] → [30|NULL] ← rear
```

The new node always goes at the back. This is why FIFO works — elements leave from the front (where the oldest element is) and enter at the rear.

Challenge 4: Code Explanation

Part 4: Array Queue vs. Linked List Queue

Feature	Circular Array Queue	Linked List Queue
Size	Fixed (`MAX`)	Dynamic
Overflow	When `count == MAX`	Only if `malloc` fails
Wasted space	None (circular)	Pointer overhead per node
Implementation	Modular arithmetic	Two pointers (front, rear)
Cache performance	Better (contiguous)	Worse (scattered nodes)
Edge case	Wrap-around logic	`rear = NULL` when empty

Both achieve O(1) for all operations. Circular array is simpler and faster; linked list is more flexible.

Lab Session Complete!

Summary: What You Built

Challenge 1: Array Stack

- Implemented push, pop, peek using a fixed-size array
- Handled overflow and underflow conditions

Challenge 2: Linked List Stack

- Built a dynamic stack with no size limit
- Practiced memory management with `malloc` / `free`

Challenge 3: Circular Queue (Array)

- Solved the "wasted space" problem using modular arithmetic
- Traced wrap-around behaviour with front and rear indices

Challenge 4: Linked List Queue

- Maintained front and rear pointers for O(1) operations
- Handled the critical single-element edge case

Key Concepts Reinforced

Stack (LIFO)

- Only the top element is accessible
- Push and pop at the same end
- Array: `top` index, Linked List: head pointer

Queue (FIFO)

- Elements enter at the rear, leave from the front
- Two access points needed for O(1) operations
- Linear arrays waste space → Circular Queue solves it

Key Concepts Reinforced (continued)

Array vs. Linked List Trade-offs

- Arrays: fixed size, better cache performance, simpler code
- Linked Lists: dynamic size, pointer overhead, need to free memory
- Both achieve O(1) for all fundamental operations

Memory Management Patterns

- `malloc` every new node, `free` every removed node
- Use `temp` pointer to save address before freeing
- Set dangling pointers to `NULL` after freeing

Common Mistakes to Avoid

Forgetting Underflow Check:

```
int pop(struct Stack* s) {
    return s->arr[s->top--];  // If top == -1, reads garbage!
    // Fix: Always check isEmpty() first
}
```

Dangling Rear Pointer:

```
q->front = q->front->next;
// If front is now NULL, rear still points to freed memory!
// Fix: if (q->front == NULL) q->rear = NULL;
```

Wrong Modular Arithmetic:

```
q->rear = q->rear + 1;         // Doesn't wrap around!
q->rear = (q->rear + 1) % MAX; // Correct!
```

Common Mistakes to Avoid (continued)

Accessing Freed Memory in Pop:

```
free(*top);
*top = (*top)->next;  // DANGER: *top is already freed!
// Fix: Save temp = *top, update *top, then free(temp)
```

Forgetting to Free Memory:

```
// After popping all elements from linked list stack...
// If you just set top = NULL without freeing nodes → memory leak!
// Fix: Use a freeStack() function that frees every node
```

Complexity Summary

Challenge	Data Structure	Enqueue/Push	Dequeue/Pop	Peek	Space
1. Array Stack	Stack (Array)	O(1)	O(1)	O(1)	O(MAX)
2. LL Stack	Stack (Linked List)	O(1)	O(1)	O(1)	O(n)
3. Circular Queue	Queue (Array)	O(1)	O(1)	O(1)	O(MAX)
4. LL Queue	Queue (Linked List)	O(1)	O(1)	O(1)	O(n)

Take-Home Challenge (Optional)

Extend your implementations with these features:

1. **Stack:** Implement a function `reverseString(str)` using your array stack to reverse a string character by character
2. **Queue:** Implement `interleaveQueue()` — given a queue of even size, interleave the first half with the second half (e.g., `[1,2,3,4,5,6]` → `[1,4,2,5,3,6]`)
3. **Bonus:** Implement a function `sortStack()` that sorts a stack using only one additional stack (no arrays allowed!)

Think about:

- What is the time complexity of each operation?
- How does the stack's LIFO property help in string reversal?
- Can you implement a queue using two stacks?